

Permessi per ruolo e per utente e Policy di autorizzazione

Architettura, modello dati, enforcement applicativo e a livello di pagina

Ambito: Tappe T1–T3 dell'autorizzazione dinamica DB-driven (ruoli, menu, permessi, policy).

Stack: .NET 10 · Blazor Web App (InteractiveServer) · Dapper · SQL Server · cookie auth · MudBlazor.

Schema DB: `fatt.*` · **Hashing:** PBKDF2 (PasswordHasher<T>).

Documento generato il: 10 giugno 2026.

Indice

1. Visione d'insieme e principi di progettazione

Il problema: autorizzazione configurabile a runtime

Le due forme di controllo: policy fisse e menu dinamico

Mappa dei componenti

2. Modello dati dei permessi

Ruoli come righe di tabella

Utenti e chiave esterna al ruolo

Struttura del menu e mapping di visibilità

Diagramma entità-relazioni

3. Identità e claim: cosa succede al login

4. Le policy ASP.NET "fisse" (RequireAdmin / RequireSuperadmin)

5. Il cuore: `MenuService` e il calcolo dell'accesso

Le regole di accesso

Risoluzione override utente / ruolo

Costruzione dell'albero e delle pagine consentite

La cache per-circuit

6. Enforcement a livello di pagina: la guardia di rotta

7. Il menu di navigazione data-driven

8. Configurazione dei permessi (UI di amministrazione)

Gestione ruoli

Permessi per ruolo

Permessi per utente (override)

La regola dell'auto-inclusione del gruppo-padre

9. Tracciamento dei livelli di difesa (riepilogo)

10. Casi limite, scelte e trade-off

1. Visione d'insieme e principi di progettazione

1.1 Il problema: autorizzazione configurabile a runtime

Il requisito non è un semplice "chi è Admin vede tutto, gli altri no". L'amministratore deve poter **creare nuovi ruoli** e **decidere a runtime**, senza ricompilare l'applicazione, quali voci di menu (e quindi quali pagine) ciascun ruolo può vedere; deve inoltre poter applicare **eccezioni per singolo utente**. Questo esclude l'approccio classico con un `enum Ruolo` cablato nel codice e con attributi `[Authorize(Roles=...)]` statici su ogni pagina.

La soluzione adottata sposta la conoscenza "chi vede cosa" **dal codice al database**: i ruoli sono righe, i menu sono righe, e la relazione "ruolo → menu visibili" è una tabella di mapping che l'admin modifica da un'interfaccia. Il codice contiene solo poche **regole fisse e non negoziabili** (i due ruoli di sistema), tutto il resto è dato.

1.2 Le due forme di controllo: policy fisse e menu dinamico

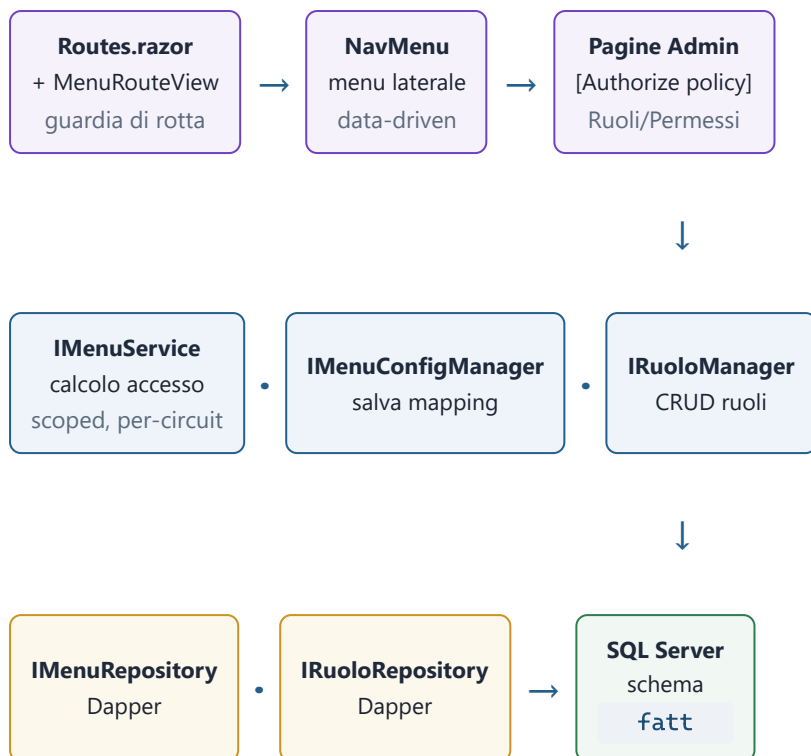
Convivono **due meccanismi complementari**, deliberatamente tenuti separati perché rispondono a domande diverse:

Meccanismo	Domanda a cui risponde	Implementazione	Su cosa agisce
Policy ASP.NET (fisse)	"Sei un amministratore?"	Policy <code>RequireAdmin / RequireSuperadmin</code> + <code>[Authorize]</code>	Le pagine di amministrazione, riservate ai ruoli di sistema
Menu dinamico + guardia di rotta	"Questo ruolo/utente ha ricevuto questa pagina?"	<code>IMenuService</code> + <code>MenuRouteView</code> (DB-driven)	Tutte le pagine di dominio, configurabili a runtime

La scelta di **non** implementare un `IAuthorizationPolicyProvider` dinamico (tecnicamente possibile ma complesso) è esplicita: con "menu dinamico + guardia di rotta" si ottiene lo stesso risultato con pochi pezzi semplici e leggibili. Sopra a tutto resta la **fallback policy** che impone l'autenticazione: chi non è loggato finisce su `/login` prima ancora che questi meccanismi entrino in gioco.

1.3 Mappa dei componenti

Il flusso rispetta l'architettura a livelli del progetto (UI → Manager → Repository → DB). I componenti coinvolti nell'autorizzazione:



Legenda colore: viola = Presentation (Blazor) · blu = Application (Manager/Service) · ambra = Data Access (Repository Dapper) · verde = Database.

Principio cardine

La UI non interroga mai il database per decidere l'accesso: chiede a **IMenuService**, che è l'unico punto in cui vivono le regole di accesso. Cambiare le regole significa toccare una sola classe.

2. Modello dati dei permessi

2.1 Ruoli come righe di tabella

I ruoli non sono un `enum` ma righe di `fatt.Ruoli`. Due righe sono "di sistema" (`IsSistema = 1`) e identificate da un **codice stabile** (`SUPERADMIN`, `ADMIN`): è il codice — non il nome visualizzato — che il programma usa per riconoscerle. Tutti gli altri ruoli sono "custom" (codice `NULL`) e completamente configurabili.

Migrations/006_RuoliUtenti.sql

```
CREATE TABLE fatt.Ruoli
(
  IdRuolo      UNIQUEIDENTIFIER NOT NULL CONSTRAINT PK_Ruoli PRIMARY KEY,
  Codice       NVARCHAR(20)      NULL,          -- 'SUPERADMIN'/'ADMIN'; NULL = custom
  Ruolo        NVARCHAR(50)      NOT NULL,       -- nome visualizzato/modificabile
  Descrizione   NVARCHAR(200)    NULL,
  IsSistema    BIT NOT NULL DEFAULT (0),        -- 1 = non eliminabile/rinominabile
  IsAttivo      BIT NOT NULL DEFAULT (1),
  CreatedAt     DATETIME2(3) NOT NULL DEFAULT (SYSUTCDATETIME()),
  UpdatedAt     DATETIME2(3) NOT NULL DEFAULT (SYSUTCDATETIME()),
  CONSTRAINT UQ_Ruoli_Ruolo UNIQUE (Ruolo)
);
-- Un solo ruolo per ciascun codice di sistema (i custom hanno Codice NULL):
CREATE UNIQUE INDEX UX_Ruoli_Codice ON fatt.Ruoli (Codice) WHERE Codice IS NOT NULL;
```

I codici di sistema sono le uniche costanti di ruolo cablate nel codice, raccolte in una sola classe:

Entities/RuoliSistema.cs

```
public static class RuoliSistema
{
    public const string Superadmin = "SUPERADMIN";
    public const string Admin      = "ADMIN";
}
```

L'entità `Ruolo` espone due proprietà derivate che incapsulano il riconoscimento dei ruoli fissi, così il resto del codice non confronta mai stringhe a mano:

Entities/Ruolo.cs (estratto)

```
public bool ESuperadmin => string.Equals(Codice, RuoliSistema.Superadmin, StringComparison.Ordinal);
public bool EAdmin      => string.Equals(Codice, RuoliSistema.Admin,      StringComparison.Ordinal);
```

2.2 Utenti e chiave esterna al ruolo

Ogni utente ha una FK `IdRuolo` verso `fatt.Ruoli` (niente enum): eredita i permessi del ruolo. La PK è un GUID UUIDv7 generato app-side; `PasswordHash` è nullable (utente "invitato", vedi il manuale dedicato all'attivazione via email).

Migrations/006_RuoliUtenti.sql (estratto)

```
CREATE TABLE fatt.Utenti
(
  IdUtente      UNIQUEIDENTIFIER NOT NULL CONSTRAINT PK_Utenti PRIMARY KEY,
  Username      NVARCHAR(80) NOT NULL,
  Email         NVARCHAR(256) NULL,
  PasswordHash  NVARCHAR(200) NULL,      -- NULL = invitato, da attivare
  IdRuolo       UNIQUEIDENTIFIER NOT NULL,
  /* ... NomeCompleto, Attivo, TemaPreferito, UltimoLoginUtc, CreatedAt, UpdatedAt ... */
  CONSTRAINT UQ_Utenti_Username UNIQUE (Username),
  CONSTRAINT FK_Utenti_Ruoli FOREIGN KEY (IdRuolo) REFERENCES fatt.Ruoli (IdRuolo)
);
```

2.3 Struttura del menu e mapping di visibilità

La struttura del menu è dati: `fatt.Menu` (voci di primo livello: gruppi o link diretti) e `fatt.SottoMenu` (figlie, puntano sempre a una pagina). Il campo `Menu / SottoMenu` **contiene il nome della classe/pagina Razor** a cui la voce punta (es. `'Anagrafiche'`): è la stessa chiave su cui la guardia di rotta confronterà `routeData.PageType.Name`. Su `fatt.Menu` è nullable: `NULL` = voce-gruppo contenitore senza pagina propria.

Colonna `SoloAdmin` (migration 012)

Aggiunta a `fatt.Menu` per marcare i gruppi accessibili solo ad Admin/Superadmin (es. "Amministrazione"). Questi gruppi sono **esclusi dalle matrici di configurazione** e nascosti ai non-admin a prescindere da qualsiasi mapping: difesa indipendente dalla configurazione.

La visibilità è espressa da **quattro tabelle di mapping** (molti-a-molti):

Tabella	Collega	Significato	Priorità
<code>fatt.MenuRuolo</code>	Menu ↔ Ruolo	Quali gruppi vede un ruolo	Primaria
<code>fatt.SottoMenuRuolo</code>	SottoMenu ↔ Ruolo	Quali sottovoci vede un ruolo	
<code>fatt.MenuUtente</code>	Menu ↔ Utente	Override per singolo utente	Sostituisce il ruolo
<code>fatt.SottoMenuUtente</code>	SottoMenu ↔ Utente	Override per singolo utente	

Migrations/007_Menu.sql (estratto: una tabella struttura + una di mapping)

```

CREATE TABLE fatt.SottoMenu (
  IdSottoMenu UNIQUEIDENTIFIER NOT NULL CONSTRAINT PK_SottoMenu PRIMARY KEY,
  IdMenu      UNIQUEIDENTIFIER NOT NULL,
  Descrizione NVARCHAR(50) NOT NULL,          -- etichetta UI
  SottoMenu   NVARCHAR(80) NOT NULL,          -- nome pagina Razor (chiave guardia)
  Icona       NVARCHAR(64) NULL,
  Ordine      INT NOT NULL DEFAULT (0),
  Attivo      BIT NOT NULL DEFAULT (1),       -- 0 = funzione non ancora implementata
  CONSTRAINT FK_SottoMenu_Menu FOREIGN KEY (IdMenu) REFERENCES fatt.Menu (IdMenu)
);

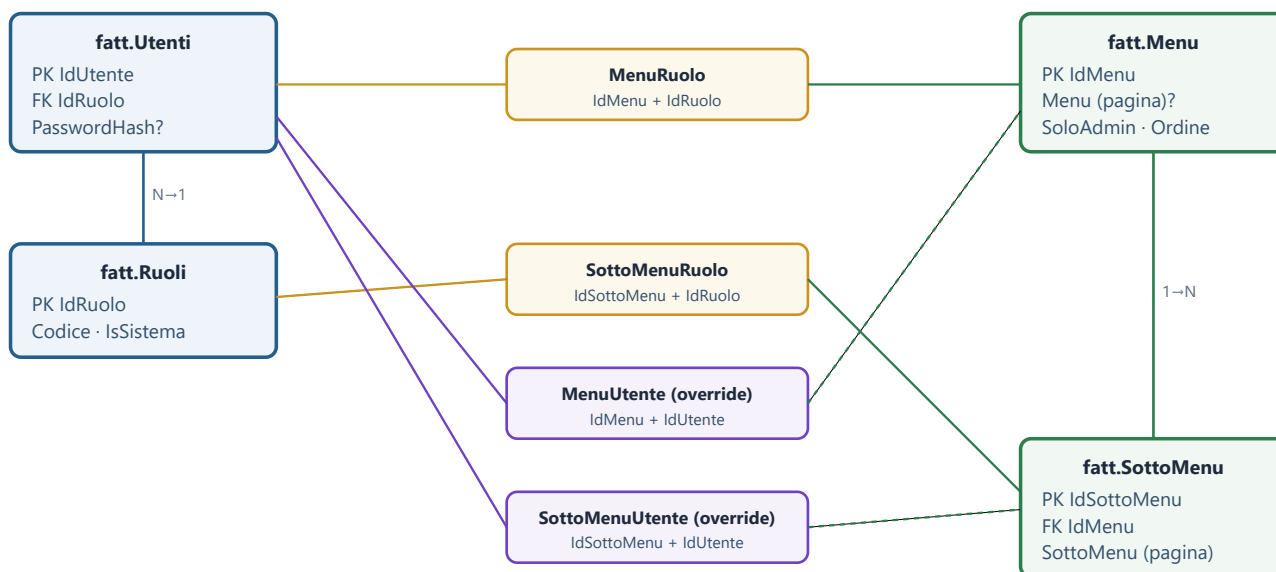
CREATE TABLE fatt.SottoMenuRuolo (
  IdSottoMenuRuolo UNIQUEIDENTIFIER NOT NULL CONSTRAINT PK_SottoMenuRuolo PRIMARY KEY,
  IdSottoMenu      UNIQUEIDENTIFIER NOT NULL,
  IdRuolo          UNIQUEIDENTIFIER NOT NULL,
  CONSTRAINT FK_SottoMenuRuolo_Sotto FOREIGN KEY (IdSottoMenu) REFERENCES fatt.SottoMenu
(IdSottoMenu),
  CONSTRAINT FK_SottoMenuRuolo_Ruolo FOREIGN KEY (IdRuolo) REFERENCES fatt.Ruoli (IdRuolo),
  CONSTRAINT UQ_SottoMenuRuolo UNIQUE (IdSottoMenu, IdRuolo)
);

```

Distinzione importante: **Attivo** ≠ **visibilità**

Attivo dice se la **funzionalità è implementata** (se 0, la voce appare in grigio, non cliccabile). La **visibilità per ruolo** è tutt'altra cosa, governata dalle tabelle di mapping. Una voce può essere visibile a un ruolo ma disabilitata perché la pagina non esiste ancora.

2.4 Diagramma entità-relazioni



Le tabelle di mapping (ambra = per ruolo, viola = override per utente) realizzano la relazione molti-a-molti tra la struttura del menu e i soggetti (ruoli/utenti). Ogni mapping ha un vincolo **UNIQUE** sulla coppia, che impedisce righe duplicate.

3. Identità e claim: cosa succede al login

L'autenticazione è a cookie. Il form di `/login` è reso server-side statico e fa POST a un endpoint minimal-API, perché solo lì `HttpContext` è "fresco" e si può chiamare `SignInAsync` (in Blazor Server interattivo la response è già committata al circuit). Verificata la password, l'endpoint costruisce i **claim** che porteranno l'autorizzazione per tutta la sessione.

Program.cs – helper condiviso di sign-in (usato da login, attivazione e reset)

```
static async Task SignInUtenteAsync(HttpContext httpContext, Utente utente,
                                   IRuoloManager ruoloManager, CancellationToken ct)
{
    var ruolo = await ruoloManager.GetByIdAsync(utente.IdRuolo, ct);

    var claims = new List<Claim>
    {
        new(ClaimTypes.NameIdentifier, utente.IdUtente.ToString()),
        new(ClaimTypes.Name, utente.Username),
        new(ClaimTypes.Role, ruolo?.Codice ?? ruolo?.Nome ?? string.Empty),
        new("id_ruolo", utente.IdRuolo.ToString()),
        new("ruolo_codice", ruolo?.Codice ?? string.Empty),
    };
    if (!string.IsNullOrEmpty(utente.NomeCompleto))
        claims.Add(new Claim("nome_completo", utente.NomeCompleto));
    claims.Add(new Claim("tema_preferito", utente.TemaPreferito));

    var identity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
    await httpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
                                  new ClaimsPrincipal(identity));
}
```

Tre claim sono il fulcro dell'autorizzazione:

Claim	Valore	A cosa serve
<code>ClaimTypes.Role</code>	Codice del ruolo (<code>SUPERADMIN</code> / <code>ADMIN</code>) o, per i custom, il nome	Far combaciare le policy ASP.NET <code>RequireRole(...)</code>
<code>ruolo_codice</code>	Solo il codice di sistema (vuoto per i custom)	<code>MenuService</code> riconosce Superadmin/Admin per il bypass
<code>id_ruolo</code>	GUID del ruolo	<code>MenuService</code> carica i mapping del ruolo dal DB

Il claim `NameIdentifier` porta l' `IdUtente` : serve a caricare gli eventuali **override per utente**. Si noti che per un ruolo custom il `Role` porta la *nome*: questo non concede alcun privilegio amministrativo, perché le policy fisse richiedono esplicitamente i codici `ADMIN` / `SUPERADMIN` .

4. Le policy ASP.NET "fisse"

Le uniche policy dichiarate a livello di framework proteggono le funzioni di amministrazione. Sono **inclusive** e centralizzate (CLAUDE.md Regola 5: le policy si dichiarano in un punto solo, non sparse nei componenti).

Authentication/AuthorizationPolicies.cs

```
public static class AuthorizationPolicies
{
    public const string RequireAdmin      = "RequireAdmin";      // Admin OPPURE Superadmin
    public const string RequireSuperadmin = "RequireSuperadmin"; // solo Superadmin (es. log errori)
}
```

Program.cs – registrazione di autenticazione e policy

```
builder.Services
    .AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options => {
        options.LoginPath      = "/login";
        options.AccessDeniedPath = "/access-denied";
        options.ExpireTimeSpan = TimeSpan.FromHours(8);
        options.SlidingExpiration = true;
    });

builder.Services
    .AddAuthorizationBuilder()
    .SetFallbackPolicy(new AuthorizationPolicyBuilder() // tutto richiede login...
        .RequireAuthenticatedUser().Build())          // ...tranne [AllowAnonymous]
    .AddPolicy(AuthorizationPolicies.RequireAdmin, p =>
        p.RequireRole(RuoliSistema.Admin, RuoliSistema.Superadmin))
    .AddPolicy(AuthorizationPolicies.RequireSuperadmin, p =>
        p.RequireRole(RuoliSistema.Superadmin));
```

La fallback policy

È il primo strato: impone l'autenticazione su **ogni** pagina. Le pagine pubbliche (login, access-denied, e le pagine dei magic-link) devono marcarsi esplicitamente con `[AllowAnonymous]` per essere raggiungibili da un utente non loggato.

Le pagine di amministrazione applicano la policy con un attributo. Esempio reale:

Components/Pages/Admin/PermessiUtente.razor (intestazione)

```
@page "/admin/permessi-utente"
@attribute [Authorize(Policy =
    ICMFatturazioni.Web.Authentication.AuthorizationPolicies.RequireAdmin)]
```

Perché due livelli per le pagine admin? Le pagine admin sono protette *sia* dalla policy ASP.NET (`RequireAdmin`) *sia* dal fatto che il loro gruppo-menu è marcato `SoloAdmin`. La policy è la difesa primaria e robusta; il flag `SoloAdmin` garantisce coerenza dell'interfaccia (il gruppo non compare e non è configurabile).

per i ruoli custom). È una doppia difesa: anche se per errore qualcuno mappasse una pagina admin a un ruolo custom, la policy la bloccherebbe.

5. Il cuore: `MenuService` e il calcolo dell'accesso

Tutta la logica "chi vede cosa" per le pagine di dominio vive in un'unica classe scoped, `MenuService`. Espone due metodi: `GetMenuVisibileAsync()` (l'albero per il NavMenu) e `PuoAccedereAsync(pageName)` (usato dalla guardia di rotta). Entrambi si appoggiano a un calcolo lazy fatto una sola volta per circuit.

5.1 Le regole di accesso

Soggetto	Cosa vede	Come
SUPERADMIN	Tutto, incluse le pagine solo-superadmin (log errori)	Bypass totale: nessuna query sui mapping
ADMIN	Tutto tranne le pagine solo-superadmin	Bypass, con eccezione esplicita
Ruolo custom	Solo le pagine mappate al ruolo... o l'override utente se presente	Lettura dei mapping dal DB
Pagine di sistema (Home, Dashboard, Login, AccessDenied, NotFound, e le pagine dei magic-link) → sempre accessibili a chiunque, non gestite a menu.		

Managers/MenuService.cs – gli insiemi fissi

```
private static readonly HashSet<string> PagineSistema = new(StringComparer.Ordinal)
{
    "Home", "Dashboard", "Login", "AccessDenied", "NotFound",
    "Attiva", "ResetPassword", "ForgotPassword", // pagine anonime dei magic-link
};
private static readonly HashSet<string> PagineSoloSuperadmin = new(StringComparer.Ordinal)
{
    "LogErrors",
};
```

5.2 Risoluzione override utente / ruolo

Il metodo privato `EnsureCaricatoAsync` legge i claim, decide il livello del soggetto e determina gli insiemi di id visibili. La logica di precedenza è: **Superadmin/Admin** → **tutto**; altrimenti **se esiste un override utente quello SOSTITUISCE il ruolo**; altrimenti si applicano i permessi del ruolo.

Managers/MenuService.cs – risoluzione visibilità (estratto)

```

var ruoloCodice = user.FindFirst("ruolo_codice)?.Value;
_isSuperadmin = string.Equals(ruoloCodice, RuoliSistema.Superadmin, StringComparison.Ordinal);
_isAdmin      = string.Equals(ruoloCodice, RuoliSistema.Admin,      StringComparison.Ordinal);
Guid.TryParse(user.FindFirst("id_ruolo)?.Value, out var idRuolo);
Guid.TryParse(user.FindFirst(ClaimTypes.NameIdentifier)?.Value, out var idUtente);

IReadOnlySet<Guid> menuVisibili, sottoVisibili;
if (_isSuperadmin || _isAdmin) {
    menuVisibili = menus.Select(m => m.IdMenu).ToHashSet();           // bypass: tutto
    sottoVisibili = sottoMenus.Select(s => s.IdSottoMenu).ToHashSet();
} else {
    var userMenu = await _menuRepository.GetMenuUtenteIdsAsync(idUtente, ct);
    var userSotto = await _menuRepository.GetSottoMenuUtenteIdsAsync(idUtente, ct);
    if (userMenu.Count > 0 || userSotto.Count > 0) {                 // override attivo
        menuVisibili = userMenu; sottoVisibili = userSotto;         // SOSTITUISCE il ruolo
    } else if (idRuolo != Guid.Empty) {
        menuVisibili = await _menuRepository.GetMenuRuoloIdsAsync(idRuolo, ct);
        sottoVisibili = await _menuRepository.GetSottoMenuRuoloIdsAsync(idRuolo, ct);
    } else { menuVisibili = new HashSet<Guid>(); sottoVisibili = new HashSet<Guid>(); }
}

```

Semantica "replace", non "unione"

L'override per utente **sostituisce** il ruolo: può sia aggiungere sia *togliere* permessi. La presenza di anche una sola riga in `MenuUtente` / `SottoMenuUtente` attiva la modalità override; rimuoverle tutte fa tornare l'utente a seguire il ruolo ("Torna al ruolo").

5.3 Costruzione dell'albero e delle pagine consentite

Dagli insiemi di id visibili, `EnsureCaricatoAsync` costruisce in un solo passaggio **due risultati**: l'albero `MenuNodo` per il `NavMenu` e l' `HashSet<string>` delle pagine consentite (le chiavi che la guardia di rotta confronterà). Qui agisce anche la difesa `SoloAdmin`.

Managers/MenuService.cs – costruzione (estratto)

```

foreach (var m in menus)
{
    // Difesa: i gruppi admin-only non compaiono mai per i non-admin.
    if (m.SoloAdmin && !_isSuperadmin && !_isAdmin) continue;

    var figli = sottoMenus
        .Where(s => s.IdMenu == m.IdMenu && sottoVisibili.Contains(s.IdSottoMenu))
        .Select(s => new MenuNode { Descrizione = s.Descrizione, PaginaRazor = s.PaginaRazor,
                                   Icona = s.Icona, Attivo = s.Attivo })
        .ToList();

    foreach (var f in figli)
        if (!string.IsNullOrEmpty(f.PaginaRazor)) pagine.Add(f.PaginaRazor);

    var menuVisibile = menuVisibili.Contains(m.IdMenu);
    var eLinkDiretto = !string.IsNullOrEmpty(m.PaginaRazor);
    // un menu compare se è un link diretto visibile OPPURE ha almeno una sottovoce visibile
    if (figli.Count == 0 && !(menuVisibile && eLinkDiretto)) continue;
    if (menuVisibile && eLinkDiretto) pagine.Add(m.PaginaRazor!);

    nodi.Add(new MenuNode { Descrizione = m.DescrizioneMenu, PaginaRazor = m.PaginaRazor,
                           Icona = m.Icona, Attivo = m.Attivo, Figli = figli });
}
_albero = nodi; _pagineConsentite = pagine; _caricato = true;

```

5.4 `PuoAccedereAsync` e la cache per-circuit

`MenuService` è registrato come **scoped**: in Blazor Server vive per la durata del circuit (la sessione interattiva dell'utente). Il calcolo è quindi fatto una sola volta e riutilizzato per tutte le navigazioni. Conseguenza pratica: **le modifiche ai permessi hanno effetto dal prossimo accesso** dell'utente (lo dichiarano anche le pagine di configurazione).

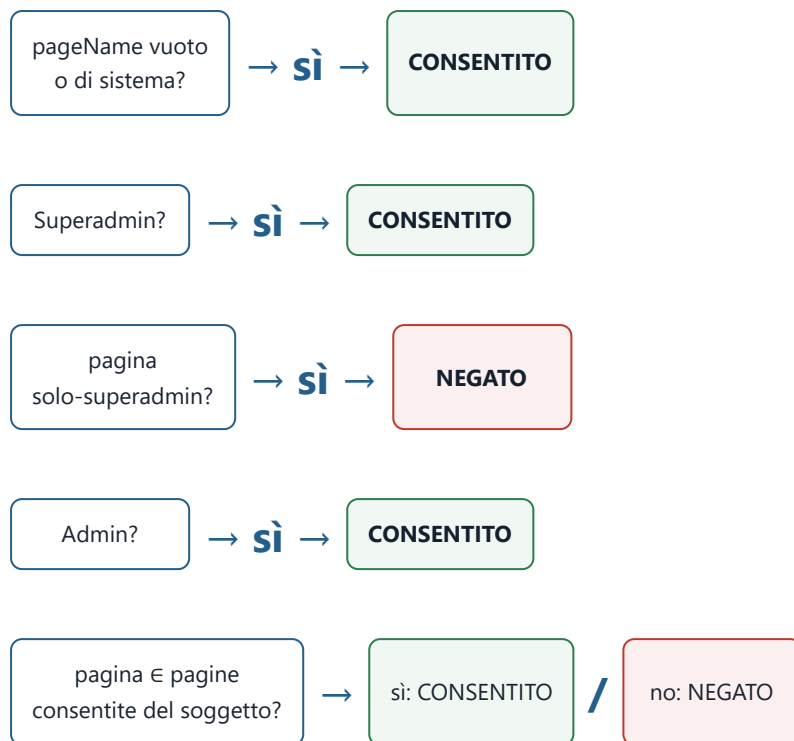
Managers/MenuService.cs – il metodo interrogato dalla guardia

```

public async Task<bool> PuoAccedereAsync(string pageName, CancellationToken ct = default)
{
    if (string.IsNullOrEmpty(pageName) || PagineSistema.Contains(pageName)) return true;
    await EnsureCaricatoAsync(ct);
    if (!_isSuperadmin) return true;
    if (PagineSoloSuperadmin.Contains(pageName)) return false; // negato ad Admin e sotto
    if (!_isAdmin) return true;
    return _pagineConsentite!.Contains(pageName);
}

```

Albero decisionale di `PuoAccedereAsync`



6. Enforcement a livello di pagina: la guardia di rotta

Il punto in cui l'accesso viene effettivamente **imposto** a ogni navigazione è un componente custom che sostituisce il `RouteView` standard. In `Routes.razor`:

Components/Routes.razor

```
<Router AppAssembly="typeof(Program).Assembly" NotFoundPage="typeof(Pages.NotFound)">
  <Found Context="routeData">
    <MenuRouteView RouteData="routeData" DefaultLayout="typeof(Layout.MainLayout)" />
    <FocusOnNavigate RouteData="routeData" Selector="h1" />
  </Found>
</Router>
```

`MenuRouteView` chiede a `MenuService` se il **nome della classe della pagina** (`routeData.PageType.Name` — la stessa chiave memorizzata nelle tabelle menu) è consentito. In caso negativo reindirizza a `/access-denied` (pagina di sistema, sempre lecita → nessun loop) e **non renderizza** la pagina richiesta.

Components/Routing/MenuRouteView.razor

```
@inject IMenuService MenuService
@inject NavigationManager Nav

@if (_consentito == true)
{
  <RouteView RouteData="RouteData" DefaultLayout="DefaultLayout" />
}

@code {
  [Parameter] public RouteData RouteData { get; set; } = default!;
  [Parameter] public Type? DefaultLayout { get; set; }
  private bool? _consentito;

  protected override async Task OnParametersSetAsync()
  {
    _consentito = await MenuService.PuoAccedereAsync(RouteData.PageType.Name);
    if (_consentito == false)
      Nav.NavigateTo("/access-denied", forceLoad: false);
  }
}
```

Tre strati di difesa, in ordine

1. **Fallback policy** (framework): non autenticato → `/login`.
2. **Guardia di rotta** (`MenuRouteView` + `MenuService`): autenticato ma la pagina non è tra quelle consentite al suo ruolo/override → `/access-denied`.
3. **Policy** `[Authorize]` sulle pagine admin: difesa robusta e indipendente per le funzioni di amministrazione (`RequireAdmin` / `RequireSuperadmin`).

Lezione appresa (regressione reale)

Una pagina anonima nuova (es. `ForgotPassword`) che **non** sia inserita in `PagineSistema` viene bloccata dalla guardia di rotta e mandata su `/access-denied`, anche se ha `[AllowAnonymous]`. La fallback policy e la guardia di rotta sono controlli *distinti*: il primo riguarda l'autenticazione, il secondo l'accesso per menu. Ogni nuova pagina pubblica va aggiunta a `PagineSistema`.

7. Il menu di navigazione data-driven

Il `NavMenu` non ha più voci cablate: chiede a `MenuService` l'albero già filtrato e lo rende. I gruppi diventano `MudNavGroup`, le foglie `NavMenuItem`. "Home" è l'unica voce fissa (pagina di sistema). Le voci con `Attivo = false` appaiono disabilitate.

Components/Layout/NavMenu.razor (estratto)

```
@inject IMenuService MenuService

@foreach (var nodo in _menu)
{
    @if (nodo.EGruppo)
    {
        <MudNavGroup Title="@nodo.Descrizione" Icon="@IconaResolver.Risolvi(nodo.Icona)"
            Expanded="@nodo.Figli.Any(f => f.Attivo)">
            @foreach (var figlio in nodo.Figli) { <NavMenuItem Nodo="figlio" /> }
        </MudNavGroup>
    }
    else { <NavMenuItem Nodo="nodo" /> }
}

@code { protected override async Task OnInitializedAsync() => _menu = await
MenuService.GetMenuVisibileAsync(); }
```

La foglia risolve due cose a partire dal solo nome memorizzato in tabella: la **rotta** (dal nome della classe pagina, via `IPageRouteResolver` che riflette gli attributi `@page` all'avvio) e l'**icona** (dal nome Material, via `IconaResolver`). Se la voce non è attiva o la rotta non si risolve, viene mostrata disabilitata.

Components/Layout/NavMenuItem.razor (estratto)

```
@if (Nodo.Attivo && _href is not null) {
    <MudNavLink Href="@_href" Icon="@_icona"
        Match="NavLinkMatch.Prefix">@Nodo.Descrizione</MudNavLink>
} else {
    <MudNavLink Disabled="true" Icon="@_icona">@Nodo.Descrizione</MudNavLink>
}

@code {
    protected override void OnParametersSet() {
        _href = Nodo.PaginaRazor is null ? null : RouteResolver.GetRoute(Nodo.PaginaRazor);
        _icona = IconaResolver.Risolvi(Nodo.Icona);
    }
}
```

Menu e guardia sono coerenti per costruzione

Lo stesso `MenuService` alimenta sia ciò che *si vede* (`NavMenu`) sia ciò a cui *si può accedere* (guardia di rotta): l'insieme delle pagine raggiungibili via menu e l'insieme delle pagine consentite sono calcolati nello stesso passaggio, quindi non possono divergere. Un utente non può "vedere" un link che non potrebbe aprire, né arrivare via URL a una pagina che il menu gli nasconde.

8. Configurazione dei permessi (UI di amministrazione)

Tre pagine, tutte protette da `RequireAdmin`, permettono di gestire ruoli e permessi senza toccare il DB a mano. La logica di scrittura passa da `IMenuConfigManager` e `IRuoloManager`; le pagine non vedono mai i repository.

8.1 Gestione ruoli (/admin/ruoli)

Una griglia con i ruoli; i ruoli di sistema sono mostrati ma **protetti** (niente icone di modifica/eliminazione) e il Superadmin è nascosto a chi non è Superadmin. Le regole di protezione sono imposte nel **manager**, non solo nascondendo i pulsanti:

Managers/RuoloManager.cs – validazioni e protezioni

```
public async Task AggiornaAsync(Guid idRuolo, string nome, string? descr, bool isAttivo,
CancellationToken ct)
{
    var esistente = await _repository.GetByIdAsync(idRuolo, ct)
    ?? throw new ArgumentException("Ruolo inesistente.", nameof(idRuolo));
    if (esistente.IsSistema) throw new RuoloProtettoException(esistente.Nome); // non rinominabile
    if (string.IsNullOrEmpty(nome)) throw new ArgumentException("Il nome del ruolo è
obbligatorio.", nameof(nome));
    if (await _repository.ExistsNomeAsync(nome, idRuolo, ct)) throw new
RuoloDuplicatoException(nome);
    await _repository.UpdateAsync(idRuolo, nome.Trim(), descr, isAttivo, ct);
}

public async Task EliminaAsync(Guid idRuolo, CancellationToken ct)
{
    var esistente = await _repository.GetByIdAsync(idRuolo, ct)
    ?? throw new ArgumentException("Ruolo inesistente.", nameof(idRuolo));
    if (esistente.IsSistema) throw new RuoloProtettoException(esistente.Nome);
    var utenti = await _repository.CountUtentiAsync(idRuolo, ct);
    if (utenti > 0) throw new RuoloInUsoException(esistente.Nome, utenti); // niente orfani
    await _repository.DeleteAsync(idRuolo, ct);
}
```

Le tre eccezioni tipizzate (`RuoloProtettoException`, `RuoloDuplicatoException`, `RuoloInUsoException`) portano messaggi user-friendly che la pagina mostra come avviso, senza loggarle come errori di sistema.

8.2 Permessi per ruolo (/admin/permessi) — la matrice ruolo×menu

Si seleziona un ruolo custom e si spuntano i menu/sottomenu che può vedere. I gruppi `SoloAdmin` sono esclusi dalla matrice (`.Where(m => !m.SoloAdmin)`); i ruoli di sistema non compaiono nel selettore (vedono tutto per regola). Al salvataggio:

Components/Pages/Admin/PermessiMenu.razor (estratto)

```
// caricamento del mapping attuale del ruolo
var mapping = await Config.GetMappingRuoloAsync(idRuolo.Value);
_menuChecked = new HashSet<Guid>(mapping.MenuIds);
_sottoChecked = new HashSet<Guid>(mapping.SottoMenuIds);
// ...
// salvataggio
await Config.SalvaMappingRuoloAsync(_selectedRuolo.Value, _menuChecked.ToList(),
_sottoChecked.ToList());
```

8.3 Permessi per utente (/admin/permessi-utente) — l'override

Stessa matrice, ma per un singolo utente con ruolo custom. La pagina rende esplicita la semantica "replace": un banner avvisa se l'utente è *personalizzato* (l'override sostituisce il ruolo) o se *segue il ruolo*; il pulsante "Torna al ruolo" rimuove l'override. Punto chiave: se non personalizzato, la matrice si **pre-compila dal ruolo**, così salvando si parte dai permessi correnti.

Components/Pages/Admin/PermessiUtente.razor (estratto)

```
_personalizzato = await Config.HasPersonalizzazioneUtenteAsync(idUtente);
var mapping = _personalizzato
    ? await Config.GetMappingUtenteAsync(idUtente)           // l'override esistente
    : await Config.GetMappingRuoloAsync(idRuolo);           // punto di partenza dal ruolo
// "Torna al ruolo":
await Config.RimuoviPersonalizzazioneUtenteAsync(idUtente); // svuota i mapping utente
```

8.4 La regola dell'auto-inclusione del gruppo-padre

Una sottovoce non sarebbe raggiungibile nel NavMenu se il suo gruppo-padre non fosse anch'esso visibile. Per questo, salvando, il manager **aggiunge automaticamente** i gruppi-padre dei sottomenu spuntati — l'admin non deve ricordarsene:

Managers/MenuConfigManager.cs

```
public async Task SalvaMappingRuoloAsync(Guid idRuolo, IReadOnlyCollection<Guid> menuIds,
                                         IReadOnlyCollection<Guid> sottoMenuIds, CancellationToken
ct)
{
    var menuFinali = await IncludiGruppiAsync(menuIds, sottoMenuIds, ct);
    await _menuRepository.SetMappingRuoloAsync(idRuolo, menuFinali, sottoMenuIds, ct);
}

private async Task<HashSet<Guid>> IncludiGruppiAsync(IReadOnlyCollection<Guid> menuIds,
                                                    IReadOnlyCollection<Guid> sottoMenuIds, CancellationToken ct)
{
    var sottoMenus = await _menuRepository.GetSottoMenusAsync(ct);
    var menuFinali = new HashSet<Guid>(menuIds);
    var sottoSelezionati = sottoMenuIds.ToHashSet();
    foreach (var s in sottoMenus)
        if (sottoSelezionati.Contains(s.IdSottoMenu)) menuFinali.Add(s.IdMenu); // aggiunge il
padre
    return menuFinali;
}
```

Il salvataggio è una **sostituzione integrale** (il repository cancella le righe di mapping esistenti per quel ruolo/utente e reinserisce quelle indicate), così non restano residui di configurazioni precedenti.

9. Tracciamento dei livelli di difesa (riepilogo)

Una singola richiesta di pagina attraversa, nell'ordine, i seguenti controlli:

#	Controllo	Dove	Esito negativo
1	Autenticazione	Cookie middleware + fallback policy	→ <code>/login</code>
2	Accesso per ruolo/menu (pagine di dominio)	<code>MenuRouteView</code> → <code>MenuService.PuoAccedereAsync</code>	→ <code>/access-denied</code>
3	Policy amministrativa (solo pagine admin)	<code>[Authorize(Policy=RequireAdmin/Superadmin)]</code>	→ <code>/access-denied</code>
4	Coerenza UI	Flag <code>SoloAdmin</code> + filtri nelle pagine di configurazione	Voce nascosta / non configurabile

I controlli 2 e 3 sono ridondanti per le pagine admin **di proposito**: la guardia di rotta dà coerenza con il menu, la policy dà la garanzia robusta lato framework.

10. Casi limite, scelte e trade-off

- **Ruolo custom senza alcun mapping** → non accede a nessuna pagina di dominio (solo le pagine di sistema). È il default sicuro: un nuovo ruolo non vede nulla finché l'admin non gli assegna qualcosa.
- **Override che toglie tutto** → un utente può essere ristretto sotto al suo ruolo. Per riportarlo al ruolo si usa "Torna al ruolo" (rimuove le righe override), non si rimettono i permessi a mano.
- **Modifiche non immediate** → poiché `MenuService` è in cache per-circuit, i nuovi permessi valgono dal prossimo accesso. È un trade-off consapevole: si evita di ricalcolare l'albero a ogni navigazione. Le pagine di configurazione lo comunicano esplicitamente all'utente.
- **Perché il claim Role porta il nome per i custom** → serve solo a popolare `ClaimTypes.Role`; non dà privilegi perché le policy richiedono i *codici* di sistema. I permessi reali dei custom passano solo dai mapping.
- **Niente `IAuthorizationPolicyProvider` dinamico** → scelta di semplicità: l'accesso dinamico è gestito dalla guardia di rotta, molto più leggibile di policy generate a runtime.
- **Sicurezza dei messaggi** → le eccezioni di validazione (ruolo protetto/duplicato/in uso) non vengono loggate come errori; gli errori imprevisti sì, tramite `ILogger` (Regola 6).

In una frase

Le **policy fisse** proteggono l'amministrazione; il **menu dinamico DB-driven**, calcolato da `MenuService` e imposto dalla **guardia di rotta**, governa tutto il resto in modo configurabile a runtime — con la stessa fonte di verità che alimenta sia ciò che si vede sia ciò a cui si può accedere.